

TDD

Etapa 1: Red. Enfoque Declarativo

Primero escribimos la prueba antes de tocar cualquier línea de código de la aplicación.

Acá nos enfocamos en el **"qué"** queremos que haga el sistema y no en el **"cómo"**.

Declaramos la intención: definimos cómo se va a llamar la función, qué le vamos a pasar y qué esperamos recibir de vuelta.

¿Cómo lo hacemos (con la función sumar)? Escribimos un test que llame a `sumar(1, 2)` y hacemos la aserción de que el resultado esperado sea `3`.

Al ejecutar esta prueba, **tiene que fallar sí o sí (ponerse en rojo)**. Va a fallar porque la función `sumar` ni siquiera existe todavía, o no retorna nada.

Si la prueba pasará de entrada sin haber escrito código, significa que la prueba está mal hecha. Que falle nos asegura que el test realmente está evaluando lo que necesitamos que evalúe y nos da el punto de partida para ir al verde.

Etapa 2: Green. Enfoque Imperativo

Vamos a escribir el mínimo código necesario para que pase la prueba (No nos enfocamos en que sea lindo).

¿Cómo lo podemos hacer (con la función sumar)? Poner un `return a + b` o `3`.

Lo ideal **sería 3**. Esto es debido a que debemos poner el resultado que esperamos.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Resultados
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    assert(resultado == 3)

Class Calculadora:
    def sumar(a,b):
        return 3
```

Un problema de gran tamaño lo debemos dividir en varias partes para que no se nos escape ninguna arista.

Lo mejor que debemos hacer es “Fakearlo”. Como ya sabemos que en la prueba suma 1 y 2, el resultado si o si va a ser 3 y por eso ponemos ese resultado. Más adelante lo generalizamos.

Etapa 3: Refactor

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 3,b = 2)
    //Resultados
    assert(resultado == 5) //nueva comparación a + b

Class Calculadora:
    def sumar(a: Number,b: Number) → Number:
        return a + b
```

Cambiamos los parámetros de entrada. Cuando ejecutamos la prueba el código me debería retornar 3, entonces va a fallar y esperaba 5.

Esto me indica que debemos generalizar. Cambiamos el código de la suma y ahora retorna a + b.

Entonces las tres etapas son:



Un ciclo es una pasada por las tres etapas. La idea de cada etapa es ir generalizando más y más el código.

Patrones

Sirven para saber pasar de una etapa a otra.

RED

1. Starter Test

- **¿Qué es?** Elegir la prueba más trivial y sencilla posible.
- **¿Para qué sirve?** El objetivo es simplemente dar el puntapié inicial para que el flujo arranque; más adelante ya nos encargaremos de complejizar.

2. One Step Test

- **¿Qué es?** El paso que sigue una vez que ya arrancamos.
- **¿Para qué sirve?** Continuar con el caso siguiente en nivel de dificultad (el siguiente más fácil), sin saltar de golpe al más complejo de todos. La idea es ir cubriendo todas las aristas y casos borde de a poco.

3. Explanation Test

- **¿Qué es?** Un test para clarificar dudas de negocio o diseño.
- **¿Para qué sirve?** Al trabajar en equipo, es común que surjan debates o discrepancias sobre cómo debería comportarse el sistema. En lugar de discutir en el aire, expresamos y acordamos ese comportamiento directamente a través de un test.

4. Learning Test

- **¿Qué es?** Un test orientado a la experimentación.

- **¿Para qué sirve?** Cuando nos tenemos que conectar con un sistema externo, librería o código ajeno, lo mejor es armar un test para "jugar" con la herramienta, experimentar y entender bien cómo funciona antes de meterla a producción.

5. Another Test

- **¿Qué es?** Un freno a tiempo para no perder el foco.
- **¿Para qué sirve?** Al resolver un problema, es normal que se nos ocurran variantes, caminos alternativos o dudas paralelas. En lugar de desviarnos de lo principal, paramos, anotamos esa nueva problemática en la lista de tests pendientes y volvemos a concentrarnos en el test actual.

6. Regression Test

- **¿Qué es?** Un test que reproduce un bug específico.
- **¿Para qué sirve?** Cuando salta un error en el sistema, escribimos un test que replique exactamente los pasos y el entorno que llevaron a esa falla. Así no solo solucionamos el bug actual, sino que nos aseguramos de que no vuelva a ocurrir nunca más en el futuro.

GREEN

1. Fake It (Falsificalo)

- **¿Qué es?** Devolver un valor hardcodeado (fijo).
- **¿Para qué sirve?** En lugar de escribir la lógica real de inmediato, devolvemos exactamente lo que la prueba espera recibir para que pase a verde ya mismo. Es el paso más rápido para validar la estructura antes de generalizar.

2. Triangulación

- **¿Qué es?** Crear dos o más casos de prueba distintos sobre el mismo comportamiento.
- **¿Para qué sirve?** Nos invita a escribir varios casos para asegurarnos de que la solución no sea sólo un valor fijo casual, sino que responda a la lógica real de la aplicación. Nos ayuda a llevar la solución de un caso particular a una solución genérica.

3. Obvious Implementation (Implementación Obvia)

- **¿Qué es?** Escribir la lógica real de una sola vez.
- **¿Para qué sirve?** Si ya sabemos perfectamente cómo resolverlo y es algo sencillo, no hace falta pasar por *Fake It* ni dar rodeos. Escribimos directamente la solución definitiva tal cual es. Suele servir para problemas triviales o soluciones muy claras.

4. One To Many (De Uno a Muchos)

- **¿Qué es?** Pasar de resolver para un elemento individual a resolver para una colección.
- **¿Para qué sirve?** Si tenemos una función que al final debe operar sobre un conjunto de datos (una lista o colección), primero nos enfocamos en hacerla funcionar pasándole una sola unidad. Una vez que eso anda perfecto, en las siguientes iteraciones la adaptamos para que maneje la colección completa.

Diseño de Test

1. Test List

- **¿Qué es?** Una lista de pendientes con todas las pruebas que planeamos realizar.
- **¿Para qué sirve?** Nos permite vaciar la cabeza y anotar todos los casos que se nos ocurran (casos felices, errores, aristas) para ir resolviéndolos uno por uno sin perder el hilo de lo que falta.

2. Isolated Test (Pruebas Aisladas)

- **¿Qué es?** Tests completamente independientes entre sí.
- **¿Para qué sirve?** Ningún test debe depender del resultado o del estado que dejó otro test anterior para funcionar. Cada prueba tiene que poder ejecutarse por su cuenta, en cualquier orden, sin afectar al resto.

3. Test First

- **¿Qué es?** Escribir la prueba antes que el código productivo.
- **¿Para qué sirve?** Es la regla fundamental: no se escribe ni una sola línea de lógica en la aplicación hasta no haber creado primero el test que falle y justifique esa implementación.

4. Assert First / Assert Test

- **¿Qué es?** Escribir la prueba empezando por el final (desde el `assert`).
- **¿Para qué sirve?** Cuando nos cuesta encarar un test por su complejidad, arrancamos directamente por el resultado esperado. A partir de esa aserción final, nos preguntamos: *"¿qué datos necesito y qué configuración previa de entorno hace falta para llegar acá?"*. Construimos el camino desde el final hacia el principio.

5. Test Data

- **¿Qué es?** Los datos de prueba utilizados en los escenarios.
- **¿Para qué sirve?** La información que usemos en los tests tiene que ser lo más simple, breve y fácil de entender posible. No usamos estructuras gigantes ni datos innecesarios si con un valor pequeño alcanza para validar la regla.

6. Evident Data (Datos Evidentes)

- **¿Qué es?** Hacer explícita la relación lógica entre las entradas y las salidas.
- **¿Para qué sirve?** Si tenemos que probar un cálculo complejo, elegimos valores donde el resultado sea visualmente obvio para cualquiera que lea el test.
 - *Ejemplo:* Si calculamos un IVA del 21%, usamos como entrada **100** para que la salida esperada sea directamente **21**, haciendo que la relación salte a la vista sin necesidad de hacer cuentas raras en la cabeza.

Hacer TDD requiere hacer refactor porque sino solamente hacemos mal código.

Refactor no te generaliza, la green es la que en algún punto hace eso. Refactor es aplicar buenas prácticas.